Where does this file live?

git for researchers, part 2: remotes, exclusions, and data

Florian Gärtner

Where we are today

	What Day 1 taught
	What Claude needs in every session
	What Claude needs only sometimes
	Your own skill
	Lunch
	A history you can go back to
→	**Where does this file live?** ← you are here
	Commit first, then let Claude move things. Then GitHub
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Decide**, for any file in your project, whether it belongs in git or in a cloud folder.
2. **Set up**, with Claude, a `.gitignore` that excludes data by default, and check it.
3. **Decide** which material may go to a commercial platform like GitHub, and which may not, even into a private repository.

The commit that was not a backup

Where we left it

Every snapshot you made is on your laptop.

The laptop is stolen, dropped, or reimaged by IT.

Your history is gone, *completely* gone, and the thoroughness of your commit discipline bought you nothing.

A remote is a second copy

A **remote** is a copy of your repository somewhere else, that you push to and pull from.

The best-known place for one is **GitHub**: a company, owned by Microsoft, that keeps repositories on its servers and shows them on a website, private or public. Git is the tool on your laptop; GitHub is one place to keep a copy. JLU GitLab is the university's own.

It does three jobs at once:

- **survival**: the copy that is not on your laptop
- **your other machine**: office desktop, laptop, the cluster
- **other people**: the co-author who needs your code, and at publication whoever checks or reuses it, such as a journal's data editor running your replication package

> **Two commands you will see Claude run**
>
> `git push` sends your commits there. `git pull` brings theirs back.

When a pull is refused

Suppose your last session at the office changed `analysis.R` and ended without a commit. At home you changed the same script on your laptop and pushed. Next morning at the office, the pull is refused, and Claude shows you git's reason:

```
error: Your local changes to the following files would be overwritten by merge:
    analysis.R
Please commit your changes or stash them before you merge.
```

Nothing is lost. Git stops before it touches a file. Only you know whether you still want yesterday's changes at this desk. So first:

> Show me what changed here since the last commit.

Claude can then set them aside (git calls this a **stash**), pull, and put them back. Where both machines changed the same lines, both versions end up in the file, and you decide which stays.

Before you leave a machine

Commit and push: your kit's `/session-end` does both, and asks before the push. At the next machine, `/startup` checks the remote.

Not everything may go to GitHub

Your own code may go to GitHub.

Not material you hold **for someone else**:

- a co-author's unpublished draft
- a manuscript you are refereeing
- licensed data extracts
- anything with personal data

None of it goes to a commercial platform, not even into a private repository.

> ⚠ **A push cannot be taken back**
>
> A commit that has been pushed has been distributed.

The alternative you have already

JLU GitLab: institutional, private by default, a remote like any other, and reachable by SSH only through the university VPN.

Connecting it is one request:

> Add `<url>` as the remote, and push main to it.

```
git remote add origin <url>
git push -u origin main
```

Claude runs the same two commands whichever remote it is. Choosing where the copy lives costs you nothing in workflow.

What does not go in

Git remembers everything, forever

That is the feature. It is also the constraint.

Commit a 2 GB dataset once, and every clone of that repository downloads 2 GB, including the clones made after you deleted the file.

> ⚠ **Deleting it later does not help**
>
> Removing a file removes it from the *current* snapshot. Every earlier snapshot still contains it, because that is what a snapshot is.

The obvious way: list what to keep out

Git reads a file called `.gitignore`. Each line names files git leaves out.

```
*.csv
*.RData
data/
output/
*.log
```

A **blocklist**: name the things to keep out.

It works until you produce a file type you did not anticipate. Which you will, on the day you are in a hurry.

Turn it around

```
*
!*/
!*.R
!*.md
!.gitignore
!**/.gitkeep
```

Line 1 excludes **everything**. Every line after it lets one thing back in.

> **The property you get**
>
> You cannot accidentally commit a file type you forgot to think about. Forgetting now means the file is *excluded*, not included.

Your kit's `/newproject` writes a `.gitignore` built this way.

The line nobody tells you about

`!*/` on line 2 lets git look inside **folders**.

Without it, line 1 excludes every folder too, and git never looks inside an excluded folder. Then `!*.R` lets back in `analysis.R` at the top of the project, but not `scripts/analysis.R`.

> ⚠ **Nothing in the state warns you**
>
> The scripts in your subfolders are simply not in the history, and you find out when a colleague clones the repository.

Keeping an empty folder

Git tracks files, not folders. An empty `output/` does not survive a clone, so the script that writes into it fails on your co-author's machine.

The fix is one request:

> Create an empty folder output/ that survives a clone.

Claude puts an empty file in it, `output/.gitkeep`.

A file whose only job is to give the folder something to hold.

Check what git leaves out

Excluding everything has a cost: a new kind of file, say your first Python script, is left out too.

When Claude adds a file by name, git refuses and says so. Claude sees that, and can let the file in with one more line in `.gitignore`, or ask you.

Files that arrive another way, copied in by you or written by a script, nobody adds by name. So before a push:

> List the files git tracks, and the files it leaves out.

Your kit's `/session-end` has Claude make this list before every push.

Where the data lives

Two locations

	Lives in	Why
Code, notes, config	git	you want every version
Data, outputs, PDFs	cloud folder	one current version is enough
A paper you write with others	cloud folder, edited in the browser	several people write the current version

No cloud folder yet? Set one up. JLU's Nextcloud gives you 100 GB for this, kept by the university.

> **The question to ask of any file**
>
> Do I want *every version of this*, or just *the current one*?

Why the repository stays out of the cloud

A cloud folder copies each file on its own. Git keeps its history in a hidden folder, `.git`, as many small files that must change together.

Put both on one folder, and they fight over the same files: the sync client adds files inside `.git`, and parts of it go missing. The repository breaks, with errors like `fatal: bad object` that say nothing about the cause.

> ⚠ **Check where your folder is**
>
> On Windows, `Documents` and `Desktop` are often synced to OneDrive. The kit's `rules.md` has Claude check the path first.

A paper you write with others

A Word or PowerPoint file you share with coauthors lives once, in the JLU Nextcloud (next.Hessenbox).

- **Edit it together in the browser.** Everyone sees the changes as they are made.
- **Not on two laptops at once.** The sync client cannot merge two edited copies: one keeps the name, the other becomes a “conflicted copy” on your laptop.
- **Claude works on your laptop’s copy.** So Claude changes the shared paper only while nobody else edits it, or writes the tables as files you paste in.

Together in the browser. On a laptop, one writer at a time.

Paths are the other half

Your script says `data/raw.csv`. On your co-author's machine the data is somewhere else entirely.

Solved by a **copy of the data in each person's project**, in a folder git leaves out: then `data/raw.csv` is right on every machine.

> ⚠ **The symptom**
>
> "It works on my machine" is almost always a hardcoded path.

Why not just put the data in git

There are tools for it: git LFS, git-annex.

For a three-person research team they usually cost more than they return: another service to configure, another failure mode, and a co-author who cannot clone the repository without help.

> **The honest rule of thumb**
>
> A shared folder everyone already has beats a correct system nobody sets up.

Time for your project's files

Exercise

Back in your project's session (`/cd` and its path). **A menu: use what helps your project now. If it does not need any of it, use the time as you see fit.**

- **Sort the files.** Ask Claude: *"Go through the files in this project. For each one, do I want every version of it, or just the current one?"* Where do you disagree, and why?
- **No cloud folder for the data yet?** Set one up in JLU's Nextcloud and keep the original data there.
- **Your project is a git repository already?** Ask Claude to list what git tracks and what it leaves out, and whether a deny-all `.gitignore` would suit it.

Block 2_6 then makes your project a repository with a remote, or connects the one you have.

Checking the remote at every start?

`/startup` checks the remote when you type it. Your kit's start hook could do it at every start instead: ask Claude to add the check to the hook.

The catch: on a laptop with a virus scanner such as Sophos, that check can make every new session wait for what feels like forever before you can type.

The better version checks once, in the first session of the day, and lets the later ones reuse the answer. **Ask me how; my own setup does it.**

What to take away

- A commit is not a backup. **A remote is.**
- Before you leave a machine: **commit and push.**
- Your own code may go to GitHub. **What you hold for someone else goes to no commercial platform.**
- Exclude by default, whitelist deliberately, **verify**.
- Every version, or just the current one? That decides where a file lives.